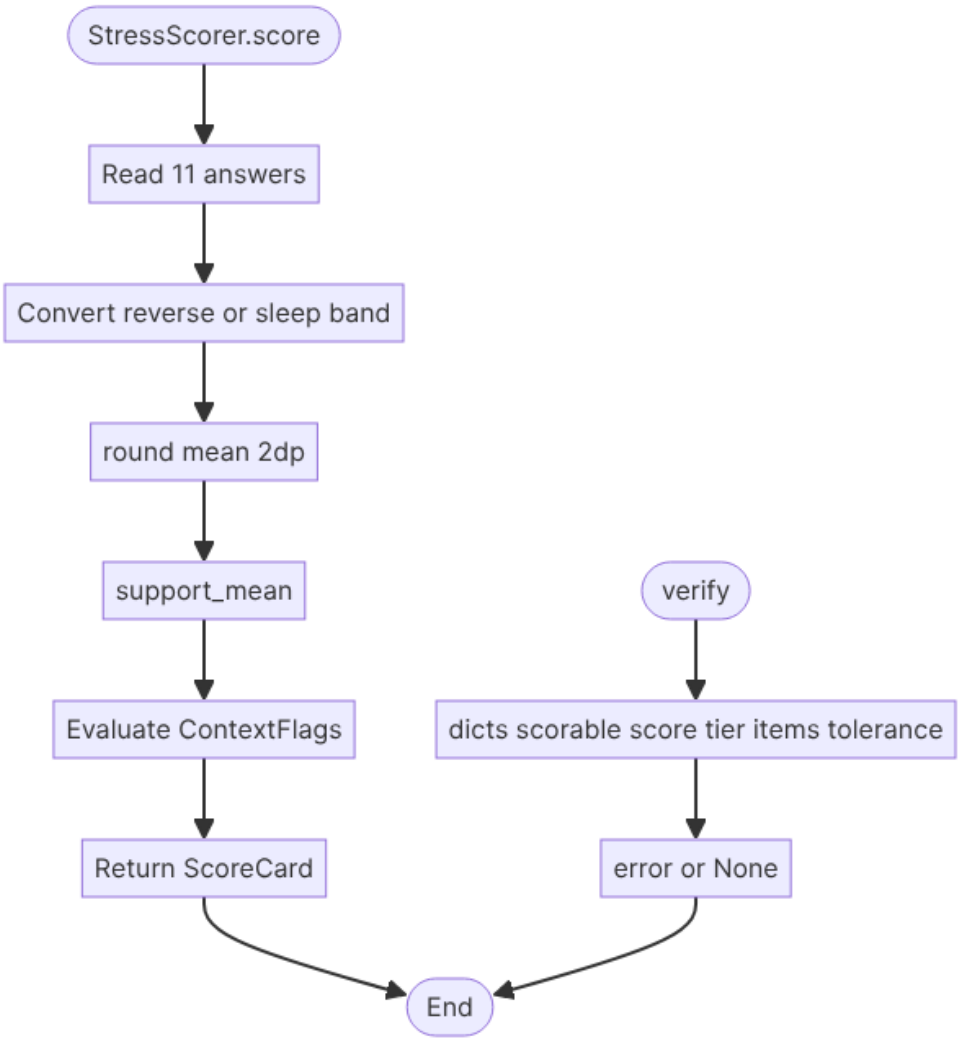
Logic Manager Layer - Part 2

Phase 2 Object-Oriented Refactoring – DoNotStress (INF1103)
Prepared by: Chan Qing Kai & ZY
Date: 8th November 2026

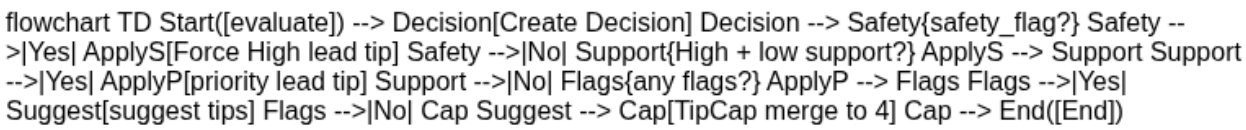
1. Flowcharts

Detailed flowcharts for the 17 Logic Manager files. Key pipeline diagrams are embedded below. Individual class .mmd sources are available separately.

1.1 StressScorer Scoring & Verification



1.2 Business Rules Pipeline


flowchart TD Start([evaluate]) --> Decision[Create Decision] Decision --> Safety{safety_flag?} Safety --
>|Yes| ApplyS[Force High lead tip] Safety -->|No| Support{High + low support?} ApplyS --> Support Support
-->|Yes| ApplyP[priority lead tip] Support -->|No| Flags{any flags?} ApplyP --> Flags Flags -->|Yes|
Suggest[suggest tips] Flags -->|No| Cap Suggest --> Cap[TipCap merge to 4] Cap --> End([End])

2. Step-by-Step Logic

answer_reader.py

Converts JSON values to finite floats; validates sleep hours (0–14) and other answers (1–5 integer). Used by StressScorer.

business_rule.py

Abstract contract: name, applies(Decision), apply(Decision). run() applies only if applies() is True. Enables OCP.

decision.py

Mutable outcome object. Starts from Gemini fields. Rules call lead_with, suggest, note, and setters for tier/label/prominence/support_priority.

score_card.py

Immutable value object holding logic_stress_score, scored_items, support_mean, risk_category, and context_flags from StressScorer.score().

__init__.py

Package marker for the logic layer.

context_flag.py

ABC for flags: name, tip_id, raised(answers, support_mean). New flags extend without changing ContextFlagRule.

context_flag_rule.py

Aggregates flags. If any raised, suggests the matching tip for each raised flag in order. Never changes tier.

support_priority_rule.py

Multi-condition rule: High category AND low_social_support in stressors → set support_priority, lead with talk_to_someone, suggest keep_social_contact.

tier_table.py

Single source of tier bands (Low <2.5, Moderate 2.5–3.5, High >3.5), labels, and prominence. Used by StressScorer and rules.

tip_cap_rule.py

Always runs. Merges unique lead tips + all Gemini tips + extras while length < 4. Caps final tips list.

trend.py

Value object for comparison (NONE/FIRST/LOWER/SIMILAR/HIGHER) and direction (NOT_ENOUGH/EASING/STEADY/BUILDING).

trend_analyser.py

Analyses StressHistory: similarity threshold 0.25; least-squares slope over window for direction. Candidate new feature.

high_answer_flag.py

Concrete ContextFlag: raised when a configured answer key >= minimum (used for workload and finances).

safety_override_rule.py

If safety_flag True: force High tier, soft_label, prominence; lead with talk_to_someone; add note. Score unchanged.

sleep_flag.py

Raised if sleep_hours_avg < 6 or sleep_quality >= 4. Suggests sleep_routine tip.

stress_scorer.py

Implements ScoreAuditor. Owns formula, conversion, ScoreCard, flags, TierTable, AnswerReader (composition). instructions() and verify() for AIManager.

support_flag.py

Raised when support_mean < 2.5. Suggests talk_to_someone.

3. Questions & Answers

Aligned with Phase 2 brief, assessment rubric, mandatory OOP concepts, and SOLID (S, O, L).

Q1: How does the Logic Manager demonstrate the Single Responsibility Principle?

StressScorer owns only the scoring formula and ScoreAuditor contract. Each BusinessRule subclass has one reason to change. ContextFlag subclasses each encode one condition. Decision is only the outcome container. LogicManager (coordinator) only sequences rules.

Q2: Where is the Open/Closed Principle applied?

New ContextFlag subclasses can be added and registered in StressScorer without changing ContextFlagRule or score(). New BusinessRule subclasses can be added to the rule list without modifying the evaluation loop.

Q3: How is the Liskov Substitution Principle satisfied?

Any BusinessRule or ContextFlag implementation can be substituted into the respective collections. Clients depend only on the abstract contracts, so behaviour remains correct.

Q4: How is the association with AIManager implemented?

StressScorer implements the ScoreAuditor ABC defined in the AI layer. AIManager holds a reference (association) and calls instructions() and verify(). The AI layer never owns scoring rules.

Q5: Which mandatory OOP concepts are demonstrated?

Abstract classes (BusinessRule, ContextFlag, ScoreAuditor), inheritance, interfaces via ABCs, polymorphism (rule and flag lists), composition (StressScorer owns reader, tiers, flags), aggregation (shared flags), encapsulation (private attributes with controlled mutation on Decision).

Q6: How does Logic preserve the four-manager architecture and AI-central pipeline?

Logic remains a distinct manager responsibility. Every record passes through AIManager first. Logic never invents outcomes on AI failure or score mismatch; it returns structured errors. The multi-condition support-priority rule operates on two AI fields.

Q7: What testing and robustness evidence would you present?

Unit tests cover StressScorer.verify(), each BusinessRule in isolation, tip-cap edge cases, safety override, and TrendAnalyser. Tests use FakeProvider or sample responses, run without a live API, and pass inside Docker.